# CSP-203: Software Tools & Technologies Lab




## Chapter – 2

## Bash Scripting: File Manipulation in Bash Demo

## (Lab Session – 8)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

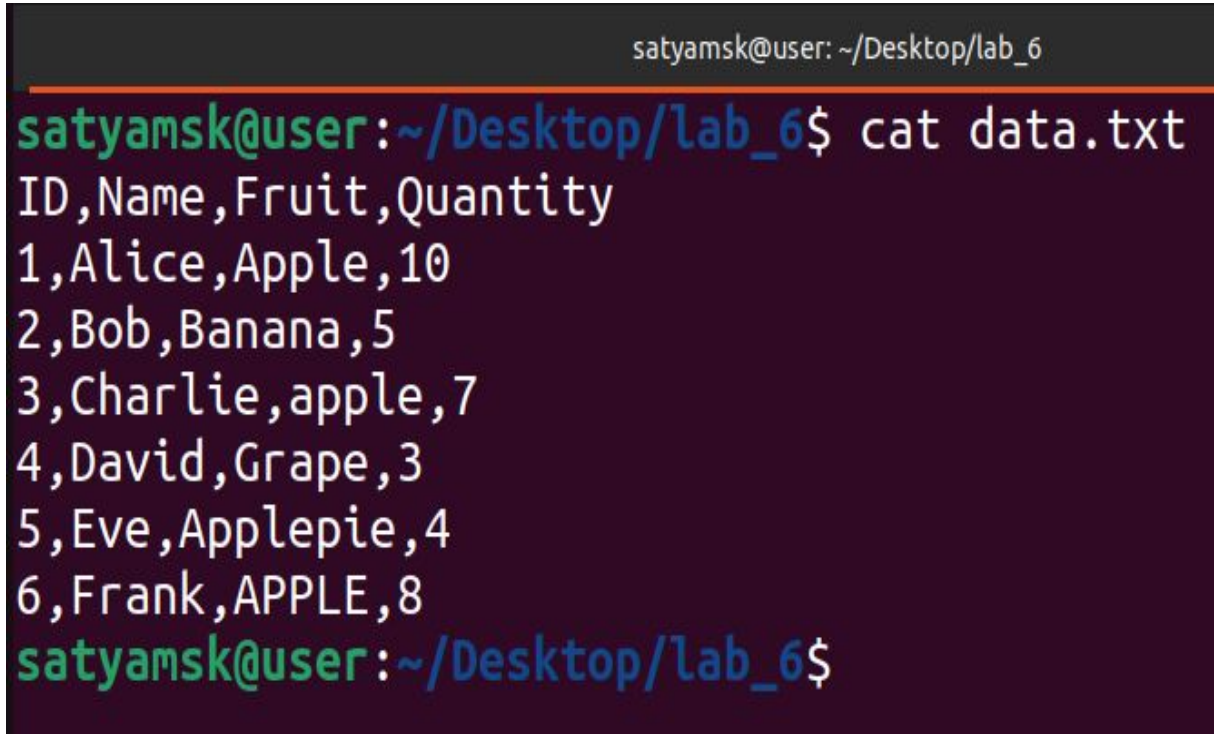
# Contents

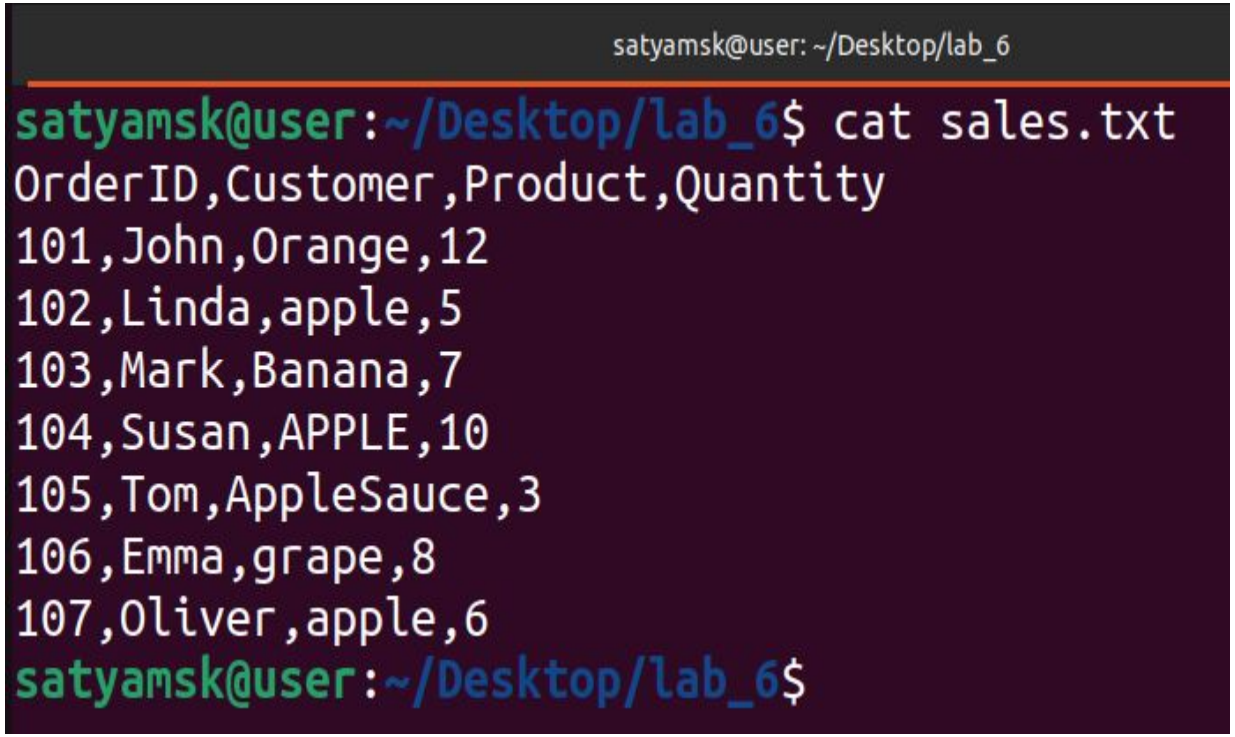
- Revisiting Text Processing Tools ( grep, Awk, sed )
- Bash Script using Text Processing tools
- Examples

# Text Processing Tool (Grep Command):

- ➔ Command: grep “string” <file_name> [Highlights all the substrings matching the given string and outputs the line]
- ➔ Command: grep -i “string” <file_name> [Ignores case matching]
- ➔ Command: grep -n “string” <file_name> [Gives the line and corresponding line number]
- ➔ Command: grep -r “string” <directory_name> [Recursively search a directory for a given string]
- ➔ Command: grep -w “string” <file_name> [Gives lines with exactly matching words, not substring]
- ➔ Command: grep -v “string” <file_name> [Give non-matching lines]
- ➔ Command: grep -m <num> “string” <file_name> [Print only first num lines with matching substring]
- ➔ Command: grep -c “string” <file_name> [Give number of lines matching substring]
- ➔ Command: grep -o “string” <file_name> [Shows only part of the matching]
- ➔ Command: grep -E “pattern” <file_name> [Basic regular expression]
- ➔ Command: grep –x “string” <file_name> [Match only whole line]

## ❑ Input Data


satyamsk@user: ~/Desktop/lab_6
satyamsk@user:~/Desktop/lab_6$ cat data.txt
ID,Name,Fruit,Quantity
1,Alice,Apple,10
2,Bob,Banana,5
3,Charlie,apple,7
4,David,Grape,3
5,Eve,Applepie,4
6,Frank,APPLE,8
satyamsk@user:~/Desktop/lab_6$


satyamsk@user: ~/Desktop/lab_6
satyamsk@user:~/Desktop/lab_6$ cat sales.txt
OrderID,Customer,Product,Quantity
101,John,Orange,12
102,Linda,apple,5
103,Mark,Banana,7
104,Susan,APPLE,10
105,Tom,AppleSauce,3
106,Emma,grape,8
107,Oliver,apple,6
satyamsk@user:~/Desktop/lab_6$

## ❑ Code

```bash
#!/bin/bash

echo "# 1. Simple search (case-sensitive)"
grep "apple" data.txt
echo

echo "# 2. Case-insensitive search"
grep -i "apple" data.txt
echo

echo "# 3. Show line numbers"
grep -n "apple" data.txt
echo

echo "# 4. Whole-word match"
grep -w "apple" sales.txt
echo

echo "# 5. Count matching lines"
grep -c "apple" sales.txt
echo

echo "# 6. Regex match"
grep -E "ap+le" data.txt
echo
```

## ❑ Output

```
satyamsk@user:~/Desktop/lab_6$ ./demo.sh
# 1. Simple search (case-sensitive)
3,Charlie,apple,7

# 2. Case-insensitive search
1,Alice,Apple,10
3,Charlie,apple,7
5,Eve,Applepie,4
6,Frank,APPLE,8

# 3. Show line numbers
4:3,Charlie,apple,7

# 4. Whole-word match
102,Linda,apple,5
107,Oliver,apple,6

# 5. Count matching lines
2

# 6. Regex match
3,Charlie,apple,7

satyamsk@user:~/Desktop/lab_6$
```

# Text Processing Tool (Awk Command):

➔ awk 'pattern { action }' filename

- pattern: A condition or regex to match lines.
- action: Commands to execute when pattern matches.
- filename: The file to process (can also use input redirection or piping).
  - $0: Entire line
  - $1, $2, ..., $n: Fields (words/columns) in the line
  - NF: Number of fields in current line
  - NR: Number of current record (line number)
  - FS: Field separator (default: space or tab)
  - OFS: Output field separator
  - RS: Record separator

## ❑ Input Data

```
satyamsk@user:~/Desktop/lab_6$ cat students.txt
101 A 85 90 78
102 B 75 88 92
103 C 90 95 89
satyamsk@user:~/Desktop/lab_6$
```

## ❑ Code

```
#!/bin/bash

echo "1. Print a specific column (Names)"
awk '{print $2}' students.txt
echo

echo "2. Print name and total marks"
awk '{total=$3+$4+$5; print $2, total}' students.txt
echo

echo "3. Use if condition (third column  > 80)"
awk '$3 > 80 {print $2, $3}' students.txt
echo

echo "4. Count number of students (records)"
awk 'END {print "Total students:", NR}' students.txt
```

## ❑ Output

```
satyamsk@user:~/Desktop/lab_6$ ./demo.sh
1. Print a specific column (Names)
A
B
C

2. Print name and total marks
A 253
B 255
C 274

3. Use if condition (third column  > 80)
A 85
C 90

4. Count number of students (records)
Total students: 3
```

# Text Processing Tool (Sed Command):

- ➔ Sed stands for Stream Editor. It is a powerful Unix utility used to search, find, replace, insert, and delete text in a stream (like a file or input from a pipeline).
- ➔ Commands:
  - ◆ Replace first match per line: sed ‘s/<old_text>/<new_text>/ <filename>
  - ◆ Replace all matches per line: sed ‘s/<old_text>/<new_text>/g’ <filename>
  - ◆ Replace all matches per line (case insensitive): sed ‘s/<old_text>/<new_text>/Ig’ <filename>
  - ◆ Delete a line with a line number: sed ‘<line_num>d’ <filename>
  - ◆ Delete a line with given expression: sed ‘/expr/d’ <filename>
  - ◆ Make the changes permanent to the file: sed -i ‘.....’ <filename>
  - ◆ Create a backup of the file: sed -i.bak ‘.....’ <filename>

## ❑ Input Data

```
satyamsk@user:~/Desktop/lab_6$ cat students.txt
101,A,85,90,78
102,B,75,88,92
103,C,90,95,89
satyamsk@user:~/Desktop/lab_6$
```

## ❑ Code

```
#!/bin/bash

# 1. Replace first occurrence of 'A' with 'Alice'
echo "1. Replace 'A' with 'Alice':"
sed 's/A/Alice/' students.txt
echo

# 2. Replace all occurrences of 'B' with 'Bob'
echo "2. Replace all 'B' with 'Bob':"
sed 's/B/Bob/g' students.txt
echo

# 3. Delete the second line
echo "3. Delete 2nd line:"
sed '2d' students.txt
echo

# 4. Print only lines containing '90'
echo "4. Print lines containing 90:"
sed -n '/90/p' students.txt
echo

# 5. Replace commas with spaces
echo "5. Replace commas with spaces:"
sed 's/,/ /g' students.txt
echo
```

## ❑ Output

```
satyamsk@user:~/Desktop/lab_6$ ./demo.sh
1. Replace 'A' with 'Alice':
101,Alice,85,90,78
102,B,75,88,92
103,C,90,95,89

2. Replace all 'B' with 'Bob':
101,A,85,90,78
102,Bob,75,88,92
103,C,90,95,89

3. Delete 2nd line:
101,A,85,90,78
103,C,90,95,89

4. Print lines containing 90:
101,A,85,90,78
103,C,90,95,89

5. Replace commas with spaces:
101 A 85 90 78
102 B 75 88 92
103 C 90 95 89

satyamsk@user:~/Desktop/lab_6$
```

Example 1: Putting it all together

- **Given Student list:**

```
arindam@ARINDAM:~$ cat database.txt
John 20 B24CS001 78
Carley 19 B24EE011 80
Shelly 21 B24ME111 94
Johnson 18 B24DS002 85
arindam@ARINDAM:~$
```

Take ID as input, if ID exists then change the marks of that student

## ❑ Code:

```
#!/bin/bash
echo -n "Enter the roll no:"
read roll_no
echo -n "Enter the updated marks: "
read marks
line_num=$(grep -n "$roll_no" database.txt| cut -d ':' -f1)
if [ -z "$line_num" ]
then
        echo "ID does not exists"
        exit 1
else
        awk -v ln=$line_num -v score=$marks 'NR==ln {$4=score} {print}' database.txt > t
emp.txt
        mv temp.txt database.txt
echo "Record Updated Successfully!!"
fi
```

## ❑ Output:

```
arindam@ARINDAM:~$ cat database.txt
John 20 B24CS001 98
Carley 19 B24EE011 80
Shelly 21 B24ME111 94
Johnson 18 B24DS002 98
arindam@ARINDAM:~$ ./demo.sh
Enter the roll no: B24ME111
Enter the updated marks: 22
Record Updated Successfully!!
arindam@ARINDAM:~$ cat database.txt
John 20 B24CS001 98
Carley 19 B24EE011 80
Shelly 21 B24ME111 22
Johnson 18 B24DS002 98
arindam@ARINDAM:~$ ./demo.sh
Enter the roll no: B24CS002
Enter the updated marks: 55
ID does not exists
arindam@ARINDAM:~$
```

Example 2:

- **Input:**

```
arindam@ARINDAM:~$ cat inputs.txt
24 55 61
55 22 11
20 54 34
arindam@ARINDAM:~$
```

Write a script to take multiple test cases consisting of three numbers and find the maximum number of each test case.

## ❑ Code:

```
#! /bin/bash

awk '{a=$1;b=$2;c=$3;
        if(a>b&&b>c){printf("a is maximum\n");}
        else if(b>c){printf("b is maximum\n");}
        else{printf("c is maximum\n");}}' inputs.txt
~
~
```

## ❑ Output:

```
arindam@ARINDAM:~$ cat inputs.txt
24 55 61
55 22 11
20 54 34
arindam@ARINDAM:~$ ./demo.sh
c is maximum
a is maximum
b is maximum
arindam@ARINDAM:~$
```


THANK
YOU